

A First Analysis of the FDR String Matcher

We extract a pseudocode of a simplified version of FDR for later analysis. A Python implementation so that we can inject instruction counter is also given in the repository. We begin with the pseudocode and comment on what is done more in the original implementation. Then we solve a specific case and try to formulate the problem in a more general way.

1 Pseudocode

The algorithm extends Shift-Or algorithm for multiple patterns. The conventions are as follows. The number of bits of a character is 8. There are 8 buckets $B_0, \dots, B_7$. Each register is of size 128 bits. We access the bit indices in a register and the character positions in a string *right to left*.

The compilation phase takes a list of patterns, put them into buckets and builds a mask for each character. The mask for a character c is a bitmask of size 128 where the left-most 64 bits are set to 0. In the right-most 64 bits, the $(8i + j)$ -th bit ($0 \leq i, j \leq 7$) is set to 0 if either

- (i) c appears in position i (indexing from the right) of a pattern in bucket B_j , or
- (ii) bucket B_j is not empty and patterns in B_j have length less than or equal to i . In this case it is called a *padding bit*.

Otherwise, the bit is set to 1.

For example, consider the buckets $[[ab, bc], [abc], [], [], [], [], [], []]$. Then the mask for each character is as follows. The padding bits are in bold.

$$\begin{aligned}
 M[a] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{10} \text{ 11111}\mathbf{11} \\
 M[b] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{10} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{10} \\
 M[c] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{10} \text{ 11111}\mathbf{11} \text{ 11111}\mathbf{01} \\
 M[x] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{10} \text{ 11111}\mathbf{11} \text{ 11111}\mathbf{11}.
 \end{aligned}$$

In the masks above, x is any character rather than a, b, c .

The execution phase takes a text and the data structure built in the compilation phase. It maintains a state mask R . It is better to interpret the bits in R as follows. Each iteration processes 8 bytes. At iteration **ITER**, the $(8i + j)$ bit ($0 \leq i \leq 15, 0 \leq j \leq 7$) is 0 means that there is currently *no contradiction* that there is a match of a pattern in bucket B_j ending at position $8 \times \mathbf{ITER} + i$. If bucket B_j has patterns of length ℓ_j , then there cannot be a match in this bucket at position less than $\ell_j - 1$. Therefore, we initialize the state mask to all 0's, except for $8i + j$ where $0 \leq i < \ell_j - 1$. After an iteration, we update $R := R \gg 64$, continuing matching the next 8 bytes. By that way, we ensure that at each position, the prefix of length at most 8 is considered.

Coming back to the example, the initial state mask is

$$R = \underbrace{00 \dots 00}_{104} \text{ 11111}\mathbf{100} \text{ 11111}\mathbf{110} \text{ 11111}\mathbf{111}.$$

Let us execution on the text *bab*c. Since there are only 4 character in the text, there is only one iteration. We give details about scanning each character as follows, where we consider the right-most 4 bytes. The newly bit set to 1 after each character is scanned is underlined.

Scan	Update	Result
<i>b</i>	$R \leftarrow R (M[b] \ll 0)$	... 11111100 111111 <u>1</u> 0 11111110 11111111
<i>a</i>	$R \leftarrow R (M[a] \ll 1)$	... 11111100 11111110 1111111 <u>1</u> 11111111
<i>b</i>	$R \leftarrow R (M[b] \ll 2)$	... 11111100 11111110 11111110 11111111
<i>c</i>	$R \leftarrow R (M[c] \ll 3)$	... 1111110 <u>1</u> 11111110 11111110 11111111

Now we report that there is a match at bucket B_0 that ends at position 1, a match at bucket B_1 that ends at position 2 and a match at bucket B_0 that ends at position 3. Since we already know the length of the patterns in each bucket, we can retrieve the beginning position of those potential matches and verify them by exact string comparison. In this case, those three potential matches are indeed matches. We give the pseudocode as in Algorithm 1 and Algorithm 2.

Algorithm 1 Compilation phase of the FDR string matcher

```

1: Assign each pattern to a bucket  $B_1, \dots, B_b$ .
2:  $M \leftarrow$  a dictionary mapping each character to a mask.
3: for  $c = 00000000$  to  $11111111$  do
4:    $M[c] \leftarrow \underbrace{00 \dots 00}_{64} \underbrace{11 \dots 11}_{64}$  ▷ Initialize.
5:   for  $j = 0$  to  $7$  and  $B_j$  is not empty do
6:      $\ell_j \leftarrow$  length of the longest pattern in bucket  $B_j$ .
7:     for  $i = \ell_j$  to  $7$  do
8:        $M[c][8i + j] \leftarrow 0$  ▷ Set padding bits to 0.
9:     endfor
10:  endfor
11: endfor
12: for  $j = 0$  to  $7$  do
13:   for each pattern  $x$  in bucket  $B_j$  do
14:     for  $i = 0$  to  $\text{length}(x) - 1$  do
15:        $M[x[i]][8i + j] \leftarrow 0$ 
16:     endfor
17:   endfor
18: endfor

```

Algorithm 2 Execution phase of the FDR string matcher

```
1:  $R \leftarrow$  a bitmask of size 128 where the  $(8i + j)$ -th bit is 0 if  $i \geq \ell_j - 1$  and 1 otherwise.
2:  $T \leftarrow$  the text to be scanned.
3: for ITER = 0 to  $\lfloor \frac{\text{length}(T)}{8} \rfloor - 1$  do
4:    $V \leftarrow T[8 \times \text{ITER} \dots 8 \times \text{ITER} + 7]$ 
5:   for  $j = 0$  to 7 do
6:      $R \leftarrow R[(M[V[j]] \ll j)]$ 
7:   endfor
8:   Report potential matches at positions where the corresponding bits in  $R$  are 0. Verify
   those potential matches by exact string comparison.
9:    $R \leftarrow R \gg 64$ 
10: endfor
```

Coming back to the buckets in the example. The current algorithm cannot give contradiction to the text ac at position 1 for bucket B_0 . *Super-characters* are introduced. Let s be the number of bits in a super-character. For example, instead of assigning $M[a]$ as in the first step of the compilation phase, we build a mask for the super-character

$$(b[0 \dots s - 8] \ll 8) | a.$$

If a character is at the end of a patterns, for example b in the pattern ab , then we assign the mask for

$$0 \ll 8 | b.$$

But now, the algorithms may *fail* to report the match of ab at bucket B_0 that ends at position 2 for the given text $abac$. The reason is that when scanning the second b , it looks and ors the mask for the super-character $(b[0 \dots s - 8] \ll 8) | c$ instead of the mask for $0 \ll 8 | b$. The idea is to set the bit(s) that may introduce a wrong contradiction to 0 in the mask for the super-character beforehand. More precisely, if a character c appears at position 0 of a pattern (that is, a bit confusing, at the end!) in bucket B_j , then we set the bit at position $8j$ to 0 in the mask for the super-character

$$(x[0 \dots s - 8] \ll 8) | c$$

for *every* character x . This can be done more efficiently, as in fact there are only 2^{s-8} such super-characters. The corrected compilation pseudocode is given in Algorithm 3. The changes or additions are highlighted in blue.

Algorithm 3 Compilation phase of the FDR string matcher

```
1: Assign each pattern to a bucket  $B_1, \dots, B_b$ .
2:  $M \leftarrow$  a dictionary mapping each character to a mask.
3:  $s \leftarrow$  the number of bits in a super-character.
4: for  $c = \underbrace{00 \dots 00}_s$  to  $\underbrace{11 \dots 11}_s$  do
5:    $M[c] \leftarrow \underbrace{00 \dots 00}_{64} \underbrace{11 \dots 11}_{64}$  ▷ Initialize.
6:   for  $j = 0$  to 7 and  $B_j$  is not empty do
7:      $\ell_j \leftarrow$  length of the longest pattern in bucket  $B_j$ .
8:     for  $i = \ell_j$  to 7 do
9:        $M[c][8i + j] \leftarrow 0$  ▷ Set padding bits to 0.
10:    endfor
11:  endfor
12: endfor
13: for  $j = 0$  to 7 do
14:   for each pattern  $x$  in bucket  $B_j$  do
15:      $f \leftarrow x[0]$  ▷ Handle the end character.
16:     for  $y = \underbrace{0 \dots 0}_{s-8}$  to  $\underbrace{1 \dots 1}_{s-8}$  do
17:        $M[(y \ll 8) | f][8j] \leftarrow 0$ 
18:     endfor
19:     for  $i = 1$  to  $\text{length}(x) - 1$  do
20:        $M[x[i]][8i + j] \leftarrow 0$ 
21:     endfor
22:   endfor
23: endfor
```

2 Analysis of a Specific Case

Let there be only one bucket and all patterns have the same length 8. This gives an intuition that the strength of the algorithms lies not mainly in parallelism but in the fact that false positives are rare. SIMD helps because there are long registers.

It is clear that the complexity of the compilation phase is $\mathcal{O}(p)$, since the number of characters to build masks are fixed.

In the execution phase, each position in the state mask R has to be or-ed at most 8 times, while 8 positions are updated at the same time. Each iteration scans 8 characters. Hence the complexity is roughly given by

$$\begin{aligned} C &= \mathcal{O}\left(\frac{n}{8}\right) \times [\text{cost of each iteration}] \\ &= \mathcal{O}(n) \times \left([\text{cost of ors}] \right. \\ &\quad + \mathbb{P}(\text{no positive}) \times 0 \\ &\quad + \mathbb{P}(\text{positive but not matched}) \times [\text{cost of rejection}] \\ &\quad \left. + \mathbb{P}(\text{positive and matched}) \times [\text{cost of confirmation}] \right). \end{aligned}$$

If hashing is used, then the cost of rejection would be less than the cost of confirmation. In our case, since we are doing exact string comparison, the cost of rejection and confirmation are in $\mathcal{O}(p)$.

Let $X_1, \dots, X_p \stackrel{iid}{\sim} U(\{0, 1\}^{64})$ be the patterns, uniformly distributed in the set of binary strings of length 64. Let $Y \sim U(\{0, 1\}^8)$ be the text at an iteration. This implies that every character is drawn independently and uniformly at random. Suppose that there is only one bucket B_0 and all patterns are in this bucket. Let A be the event that there is a match ending at a position and B be the event that there is a positive. Then $A \subset B$. Let D be the event that all patterns are distinct. We rewrite the complexity as follows.

$$\begin{aligned} C &= \mathcal{O}(n)(1 + \mathbb{P}(B^c \mid D) \times 0 + \mathcal{O}(p) \times (\mathbb{P}(A, B \mid D) + \mathbb{P}(A^c, B \mid D))) \\ &= \mathcal{O}(n)(1 + \mathcal{O}(p) \times (\mathbb{P}(A \mid D) + \mathbb{P}(B \mid D) - \mathbb{P}(A \mid D))) \\ &= \mathcal{O}(n)(1 + \mathcal{O}(p) \times \mathbb{P}(B \mid D)). \end{aligned}$$

We check that $\mathbb{P}(B^c \mid D) + \mathbb{P}(A, B \mid D) + \mathbb{P}(A^c, B \mid D) = 1$, so the formulation should be correct. The algorithm is efficient when $\mathbb{P}(B \mid D)$ is negligible. In the first time, I tried to compute $\mathbb{P}(B \mid D)$ directly. Then I realized that it is not much different if we do not condition on D . Let me give this computation later. Maybe it can be used for more general cases. In fact, $\mathbb{P}(B \mid D)$ is close to $\mathbb{P}(B)$. We have

$$\begin{aligned} \mathbb{P}(B \mid D) - \mathbb{P}(B) &= \frac{\mathbb{P}(B, D)}{\mathbb{P}(D)} - \mathbb{P}(B, D) - \mathbb{P}(B, D^c) \\ &\leq \mathbb{P}(B, D) \left(\frac{1}{\mathbb{P}(D)} - 1 \right) \\ &\leq \mathbb{P}(D) \left(\frac{1}{\mathbb{P}(D)} - 1 \right) \\ &= 1 - \mathbb{P}(D) = \mathbb{P}(D^c). \end{aligned}$$

Now we bound $\mathbb{P}(D^c)$. This is the probability that there are two patterns that are the same. Let $N = 2^{64}$. Firstly, we choose 2 among p patterns and set them to be one of the N possible patterns. Then we choose the remaining $p - 2$ patterns from the remaining $N - 1$ possible patterns. There may be some over-counting, so we have

$$\mathbb{P}(D^c) \leq \frac{\binom{p}{2} \times N \times N^{p-2}}{N^p} = \frac{\binom{p}{2}}{N}.$$

This is indeed negligible because N is very large. Now we compute $\mathbb{P}(B)$, the probability that there is a positive. Since Y is uniformly distributed and independent of the patterns, we can fix $Y = y$. For each position i from 0 to 7, there are $2^8 = 256$ possible characters. But $y[i]$ forbids one of them. Hence there are 255 available possible characters allowed for the patterns. Therefore,

$$\mathbb{P}(B) = \left(1 - \left(\frac{255}{256} \right)^p \right)^8.$$

When $p \geq 355$, $\mathbb{P}(B) \geq 0.1$. The algorithm starts to be inefficient.

Let us compute $\mathbb{P}(B \mid D)$ directly. Again, since Y is uniformly distributed and independent of the patterns, we can fix $Y = 0^{64}$. Let $U = \{0, 1\}^{64}$. Define the sets

$$S_j = \{x \in U \mid x[j] = 0\}, j \in \{0, \dots, 7\}.$$

Then B is the event that there is at least one pattern in each S_j i.e. $\{X_1, \dots, X_p\}$ hits all S_j 's. Then B^c is the event that there is some j such that $\{X_1, \dots, X_p\}$ does not hit S_j .

$$B^c = \bigcup_{j=1}^8 E_j = \bigcup_{j=1}^8 \{X_1 \notin A_j, \dots, X_p \notin A_j\}.$$

Now we use inclusion-exclusion

$$\begin{aligned}
\mathbb{P}(B \mid D) &= 1 - \mathbb{P}\left(\bigcup_{j=1}^8 E_j \mid D\right) \\
&= 1 - \sum_{\emptyset \neq S \subset [8]} (-1)^{|S|+1} \mathbb{P}\left(\bigcap_{j \in S} E_j \mid D\right) \\
&= \sum_{S \subset [8]} (-1)^{|S|} \mathbb{P}\left(\bigcap_{j \in S} E_j \mid D\right),
\end{aligned}$$

where $\bigcap_{j \in \emptyset} E_j = \Omega$. Now we can count. There are $\binom{N}{p}$ ways to choose p distinct patterns. To make $\bigcap_{j \in S} E_j$ happen we can only choose 255 possible patterns for each $j \in S$ and 256 possible patterns for each $j \notin S$. Therefore, there are

$$256^{8-|S|} \times 255^{|S|} = N \left(\frac{255}{256}\right)^{|S|}$$

possible patterns that can be chosen for each $j \in S$ and $j \notin S$. Hence there are $\binom{N(255/256)^{|S|}}{p}$ ways to choose p distinct patterns that make $\bigcap_{j \in S} E_j$ happen.

Hence, $\mathbb{P}(\bigcap_{j \in S} E_j \mid D) = \frac{\binom{N(255/256)^{|S|}}{p}}{\binom{N}{p}}$. For each $s \in \{0, \dots, 8\}$, there are $\binom{8}{s}$ subsets of $[8]$ of size s . Hence,

$$\mathbb{P}(B \mid D) = \sum_{s=0}^8 (-1)^s \binom{8}{s} \frac{\binom{N(255/256)^s}{p}}{\binom{N}{p}} \approx \sum_{s=0}^8 (-1)^s \binom{8}{s} \left(\frac{255}{256}\right)^{sp} = \mathbb{P}(B).$$

3 Problem Statement

In general, problem is stated as follows. Let $X_1, \dots, X_p \stackrel{iid}{\sim} \mathcal{D}_P$ be the patterns and $Y \sim \mathcal{D}_T$ be the text. Let the domain of the patterns be $\mathcal{X}$ and the domain of the text be $\mathcal{Y}$.

An assignment algorithm is a function $f : \bigcup_{p \in \mathbb{N}} \mathcal{X}^p \rightarrow \{0, \dots, 7\}$.